

Pruebas de Caja Negra

Proyecto BATTLESHIP – Fundamentos de Ingeniería del Software

Curso 2024/2025

En este documento se presentan las pruebas de caja negra realizadas sobre los métodos críticos del sistema BATTLESHIP, centradas en las clases controladoras del proyecto.

Se han empleado las técnicas de **partición de equivalencia** y **análisis de valores límite**, y se han documentado las clases de equivalencia y casos de prueba relevantes para cada método.

Los métodos que no presentan lógica condicional o retorno observable, como los basados en persistencia directa con Hibernate, se han excluido justificadamente de las pruebas de caja negra, siendo considerados para pruebas de integración.

isValidName(String name) — AuthController

Valida que un nombre de usuario cumple restricciones de formato, longitud, y no esté en la lista negra.

Se aplica partición de equivalencia y valores límite.

Se listan las clases y casos a continuación.

Particiones de equivalencia:

Entrada esperada	Clase	Tipo	Ejemplo
null	N1	Inválida	null
Longitud menor a 3	N2	Inválida	"ab"
Longitud exactamente 3	V1	Válida	"abc"
Longitud máxima 10	V2	Válida	"abcdefghij"
Longitud mayor que 10	N3	Inválida	"abcdefghijk"
Caracteres no alfanuméricos	N4	Inválida	"player!"
Nombre en lista negra	N5	Inválida	"admin"
Nombre correcto típico	V3	Válida	"Player1"

Casos de prueba:

ID	Propósito	Entrada	Clase Equivalente	Salida Esperada	Tipo
CP-ISNAME-01	Validar que el nombre null no es válido	null	N1 - Nulo	false	Negativa
CP-ISNAME-02	Validar que nombres con menos de 3 caracteres son inválidos	"ab"	N2 - Menor a longitud min	false	Negativa
CP-ISNAME-03	Validar que un nombre de longitud exacta 3 es válido	"abc"	V1 - Longitud mínima	true	Positiva
CP-ISNAME-04	Validar que nombres mayores a 10 caracteres son inválidos	"abcdefghijk"	N3 - Mayor a longitud max	false	Negativa
CP-ISNAME-05	Validar que nombres con caracteres no alfanuméricos no son válidos	"play@er"	N4 - Caracteres inválidos	false	Negativa
CP-ISNAME-06	Validar que un nombre presente en la blacklist no es válido	"admin"	N5 - Lista negra	false	Negativa
CP-ISNAME-07	Validar que un nombre válido, típico y permitido pase	"Player1"	V3 - Nombre válido general	true	Positiva

loginOrRegister() — AuthController

Método que gestiona el proceso de login o registro, incluyendo validaciones mediante LDAP, disponibilidad del nombre y condiciones de validez del mismo.

Se analizan combinaciones de flujo de control: login exitoso, creación con errores y cancelación.

Particiones de equivalencia:

Condición Entrada	Clase Equivalente	Tipo	Ejemplo Simulado
LoginLDAP() devuelve null	N1 - Error LDAP	Inválida	id = null
LoginLDAP() devuelve ID de usuario ya creado	V1 - Usuario existe	Válida	id = "user1" (ya existe)

Condición Entrada	Clase Equivalente	Tipo	Ejemplo Simulado
LoginLDAP() devuelve ID nuevo	V2 - Usuario nuevo	Válida	id = "user2" (no existe)
Nombre ya en uso	N2 - Nombre repetido	Inválida	"Juan" ya usado
Nombre inválido por formato	N3 - Nombre inválido	Inválida	"@", "ab", etc.
Nombre válido	V3 - Nombre válido	Válida	"David"
Nombre en blacklist	N4 - Blacklist	Inválida	"admin"
Usuario introduce null como nombre	N5 - Registro cancelado	Inválida	name = null

Casos de prueba:

ID	Propósito	Entradas simuladas	Clases Usadas	Salida Esperada	Tipo
CP-LOGIN-01	LDAP falla → no se permite login ni registro	LoginLDAP() = null	N1	false	Negativa
CP-LOGIN-02	Usuario ya existe (login correcto)	LoginLDAP() = "id1", user "id1" ya en lista	V1	true	Positiva
CP-LOGIN-03	Usuario nuevo, primer nombre repetido, segundo nombre válido	LoginLDAP() = "id2", nombres = ["Juan", "Leo"]	V2, N2, V3	true	Positiva
CP-LOGIN-04	Usuario nuevo, primer nombre inválido, segundo válido	LoginLDAP() = "id3", nombres = ["ab", "Carlos"]	V2, N3, V3	true	Positiva
CP-LOGIN-05	Usuario nuevo, nombre en blacklist	LoginLDAP() = "id4", nombres = ["admin", "Mateo"]	V2, N4, V3	true	Positiva
CP-LOGIN-06	Nuevo usuario cancela el registro (introduce null)	LoginLDAP() = "id5", nombre = null	V2, N5	false	Negativa

addUser(User user) — UserController

El método permite registrar un nuevo usuario en el sistema, asociándolo a su ID como clave. Se evalúa mediante caja negra su comportamiento frente a entradas válidas, entradas duplicadas y valores nulos. A continuación se muestran las clases de equivalencia definidas y los casos de prueba.

Particiones de equivalencia:

Condición Entrada	Clase Equivalente	Tipo	Ejemplo
Usuario válido, con ID único	V1	Válida	User("id123", "Pepe", false)
Usuario con ID ya existente	V2	Válida*	User("id123", "Juan", false)
Usuario con ID nulo	N1	Inválida	User(null, "Pedro", false)
Usuario nulo	N2	Inválida	null

Casos de prueba:

ID	Propósito	Entrada	Clase	Salida esperada	Tipo
CP-ADDUSER-01	Añadir un usuario con ID único y datos válidos	User("id1", "Pepe", false)	V1	Usuario añadido, users.size() aumenta	Positiva
CP-ADDUSER-02	Añadir un usuario con un ID ya existente (sobrescribe)	User("id1", "Carlos", false)	V2	Usuario sobrescribe anterior	Positiva
CP-ADDUSER-03	Añadir un usuario con ID nulo	User(null, "Pedro", false)	N1	NullPointerException o fallo esperado	Negativa
CP-ADDUSER-04	Añadir un usuario siendo null	null	N2	NullPointerException esperado	Negativa

deleteUser(User user) — UserController

El método elimina un usuario del mapa de usuarios cargado en memoria. Se prueba mediante caja negra su comportamiento ante usuarios válidos, no existentes y nulos.

A continuación se detallan las clases de equivalencia y los casos de prueba aplicados.
Clases de equivalencia

Particiones de equivalencia:

Condición Entrada	Clase Equivalente	Tipo	Ejemplo
Usuario existente en el mapa	V1	Válida	User("id1", "Pepe", false)
Usuario no existente	N1	Inválida	User("id2", "Otro", false)
Usuario nulo	N2	Inválida	null

Casos de prueba:

ID	Propósito	Entrada	Clase	Salida esperada	Tipo
CP-DELUSER-01	Eliminar usuario que existe en el sistema	User("id1", "Pepe", false)	V1	Usuario eliminado del mapa	Positiva
CP-DELUSER-02	Intentar eliminar usuario no existente	User("id2", "Otro", false)	N1	No se lanza excepción, sin efecto	Negativa
CP-DELUSER-03	Intentar eliminar un usuario null	null	N2	NullPointerException esperada	Negativa

realizarTurnoUser(char[][] tableroEnemigo) — *UserController*

Se trata de un método interactivo que valida entradas del usuario sobre un tablero bidimensional. Las condiciones de prueba incluyen entradas nulas, límites del tablero y casillas previamente atacadas.

A continuación se muestran las clases de equivalencia y los casos de prueba asociados.

Particiones de equivalencia:

Condición Entrada	Clase Equivalente	Tipo	Ejemplo
Tablero nulo	N1	Inválida	null
Tablero vacío	N2	Inválida	new char[0][0]
Coordenadas fuera del tablero	N3	Inválida	[5, 5] en tablero de 3x3

Condición Entrada	Clase Equivalente	Tipo	Ejemplo
Coordenadas correctas pero ya atacadas	N4	Inválida	coordenada con 'X' o 'O'
Coordenadas correctas y válidas ('?')	V1	Válida	[1, 1] si tablero[1][1] = '?'

Casos de prueba:

ID	Propósito	Entrada	Clase(s)	Salida esperada	Tipo
CP-TURNO-01	Tablero nulo	null	N1	Lanza IllegalArgumentException	Negativa
CP-TURNO-02	Tablero vacío	new char[0][0]	N2	Lanza IllegalArgumentException	Negativa
CP-TURNO-03	Usuario introduce coordenadas fuera de rango, luego válidas	tablero 3x3, entradas: [5,5] → [1,1]	N3 → V1	[1, 1]	Positiva
CP-TURNO-04	Usuario elige casilla ya atacada, luego válida	tablero[0][0] = 'O', luego elige [1,1]='?'	N4 → V1	[1, 1]	Positiva
CP-TURNO-05	Usuario elige directamente una casilla válida	tablero[2][2]='?' y elige [2,2]	V1	[2, 2]	Positiva

askMachine() — *UserController*

Este método solicita al usuario elegir el nivel de dificultad. Se valida la entrada comparando con una lista fija de valores permitidos. Si no es válida, se repite la pregunta.

Se muestra a continuación la tabla de clases de equivalencia y los casos de prueba aplicados.

Particiones de equivalencia:

Entrada	Clase	Tipo	Ejemplo
Entrada válida (FACIL/NORMAL/DIFICIL)	V1	Válida	"FACIL"
Entrada inválida	N1	Inválida	"EXPERTO", ""

Casos de prueba:

ID	Propósito	Entrada simulada	Clase	Salida esperada	Tipo
CP-MACHINE-01	Entrada válida directamente (sin repetir)	"normal"	V1	"NORMAL"	Positiva
CP-MACHINE-02	Entrada inválida, luego válida	"hard" → "DIFICIL"	N1 → V1	"DIFICIL"	Positiva

startScoresUserProcess() — *UserController*

El método startScoresUserProcess() gestiona el acceso a los submenús de puntuaciones del usuario. Su comportamiento varía según el tipo de usuario (admin o no admin) y la opción seleccionada en el submenú.

Desde el punto de vista de la caja negra, se analiza el sistema como una unidad funcional completa, ignorando su implementación interna. Las pruebas se centran en las entradas que puede introducir el usuario (selección de menú) y el tipo de usuario que está autenticado.

La técnica aplicada es partición de equivalencia, considerando como condiciones de entrada:

- Tipo de usuario (admin / no admin)
- Opción seleccionada (0, 1, 2, otras)

Se verifican los resultados esperados, como la visualización correcta de las puntuaciones o mensajes de error por falta de permisos o entradas inválidas.

Clase DataController

Los métodos de DataController se basan en llamadas directas a Hibernate para operaciones CRUD. Al no presentar lógica interna ni variaciones de salida según la entrada, su validación se realiza mediante pruebas de integración con base de datos de pruebas, y no mediante pruebas de caja negra clásicas.

